

Recognition of Boxing Punches on an Arduino Nano 33 BLE Sense

A TinyML System that Turns Gestures into Keyboard Keys

Alan Masutti - 266086
alan.masutti@studenti.unitn.it
University of Trento
Trento, Italy

Abstract

This report describes a small machine learning system that runs completely on an Arduino Nano 33 BLE Sense. The board collects data using the IMU present on it, decides which boxing punch the user has thrown, and sends the answer to a computer as a keyboard key press over USB. Four punches are recognised, the jab, the hook, the overhand and the uppercut and a fifth class called *unknown* is used to reject every other movement. The key idea of the work is that the board compute the classification directly without the need to send the raw samples to the PC for classification. The Arduino board computes a set of 42 statistical and frequency features instead of using raw data to achieve better performances. The trained network is small, 6900 bytes in TensorFlow Lite format, and reaches an accuracy of 0.84 on the test set.

Keywords

TinyML, gesture recognition, inertial sensors, embedded machine learning, Arduino, TensorFlow Lite

1 Introduction

Small microcontrollers are now fast enough to run a neural network, and this opens the possibility of moving the recognition of a movement directly on the device that is worn by the user. The advantage is that no raw sensor data has to leave the board, the answer is available in a few milliseconds, and the whole system fits in a device that may run on a battery.

The user wears an Arduino Nano 33 BLE Sense [1] on the hand, throws a punch, and the board answers with the name of the punch (using Keyboard pressing HID). Four punches are covered, the jab, the hook, the overhand and the uppercut. A fifth class, called *unknown*, collects movements that are not punches and is used to reject them, so that a random movement of the arm does not produce an answer.

To make the result immediately useful, the recognised punch is not only printed on the serial port. The board also presents itself to the host computer as a USB keyboard and taps one key of the WASD group for every punch. In practice the board becomes a small game controller that is driven by real punches.

The work started from the gesture recognition tutorial written for the same board [4], but the gestures, the preparation of the data, the features, the network and the output interface were all changed.

2 System Overview

The system has three stages.

- (1) **Data collection.** The board records windows of movement and prints one line of features per window on the serial

port. A Python script on the computer saves these lines into a CSV file, one file per punch.

- (2) **Training.** A Jupyter notebook on the computer reads the CSV files, trains a small neural network with TensorFlow and Keras, converts it to TensorFlow Lite and writes it out as a C header file.
- (3) **Inference.** The same firmware used for the data collection, compiled in a different mode, loads that header and runs the network on the board with TensorFlow Lite Micro [2], then sends the key press.

3 Data Acquisition and Feature Extraction

A punch is a short and strong movement, so the board does not sample all the time. It keeps reading the accelerometer and waits until the sum of the absolute values of the three axes passes a threshold of 2 g. This threshold was lowered with respect to the original tutorial, because with a higher value some of the slower punches were not detected.

Once the threshold is passed, the board fills a window of 128 samples. For every sample it reads the three axes of the accelerometer and the three axes of the gyroscope, and it also stores the time of the sample. The values are then scaled from the sensor range into the 0-1 range: the accelerations are collected from the interval between -4 g and $+4$ g, and the angular speeds from the interval between -2000 and $+2000$ degrees per second.

The timestamps are used to estimate the real sampling frequency of the window. The sensor does not deliver samples at a perfectly constant rate, and the frequency is needed by the power spectral density, so it is measured from the average distance between two samples instead of being assumed.

3.1 Why features and not raw samples

The first attempt followed the classic tutorial and gave the 768 raw values of the window directly to the network. That model worked well in the classroom example with two very different gestures, but it was not able to learn the four punches. The reason is probably that the raw signals are noisy and that the same punch is never thrown twice in the same way, so the network cannot find a stable pattern in a small dataset. The window is therefore summarised by a small set of features before it reaches the network.

3.2 The 42 features

Seven features are computed for each of the six axes, which gives 42 values in total. Five of them describe the signal in time and two describe it in frequency:

- the mean, the standard deviation, the minimum and the maximum of the window;

- the root mean square, defined as

$$\text{RMS} = \sqrt{\frac{1}{N} \sum_{i=0}^{N-1} x_i^2}, \quad (1)$$

- the mean and the maximum of the power spectral density, which is obtained from the fast Fourier transform $X(f)$ of the window as

$$\text{PSD}(f) = \frac{|X(f)|^2}{N f_s}, \quad (2)$$

where N is the length of the window and f_s is the sampling frequency measured above.

The transform is computed with the *arduinoFFT* library [3], and only the first half of the spectrum is used, because the signal is real and the second half carries the same information.

3.3 Why the features are computed on the board

During the development the features were first computed in Python, on the raw data sent by the board. This looked simpler, but it created a problem that took a long time to find: the values produced by the Python code and the values produced by the C code were not the same, so a model that looked good on the computer did not work once it was loaded on the board.

The solution was to move the feature extraction completely on the device and to use one single implementation for both modes. The data collection firmware prints exactly the same 42 numbers that the inference firmware puts into the input tensor of the model. There is no longer any way for the two sides to drift apart, and the CSV files are already in the format that the training needs.

4 Dataset

The dataset was recorded by a single person wearing the board on the hand. It contains 19 windows for each of the four punches and 23 windows for the *unknown* class, so 99 windows in total, and every window is one row of 42 features.

A fifth punch, the cross, was recorded as well but it was later removed from the dataset. The reason is that the cross and the jab are very close to each other, since both are straight punches thrown forward, and the features of the two classes overlap so much that the model could not separate them in a reliable way. Keeping the cross made the accuracy of the jab worse as well, so the class was dropped and its file was left in the repository in a disabled state.

The *unknown* class has a different role. It does not describe a punch, it collects normal movements of the arm that pass the acceleration threshold without being a punch. Thanks to this class the system has a way to say that nothing interesting happened, instead of being forced to choose one of the four punches every time the board is moved.

Collect the data using a script. Before training the NN, the dataset should be sampled. For this purpose the `scripts/collect_data.py` is provided, it opens the serial port and writes, it prompts on the shell what gesture to do, and sample the same amount of data for each gesture by saving the results in different files.

5 Model and Training

The network is a small fully connected classifier. It takes the 42 features as input and has three layers: a dense layer with 20 units and ReLU activation, a dense layer with 12 units and ReLU activation, and a dense output layer with 5 units and softmax activation, one unit per class. Softmax is used because only one class can be true at a time.

The model is trained with the Adam optimiser and the categorical cross entropy loss, for 300 epochs and with a batch size of 8. A batch of one sample made the training too noisy, while 8 gave a smooth and fast convergence on such a small dataset. The 99 windows are shuffled and then split into 60% for training, 20% for validation and 20% for testing, which gives 59, 21 and 19 windows.

Figure 1 shows the training and the validation loss. The training loss keeps going down, while the validation loss stops improving much earlier. This distance between the two curves is the sign that the model is starting to memorise the training set, which is expected with a dataset of this size.

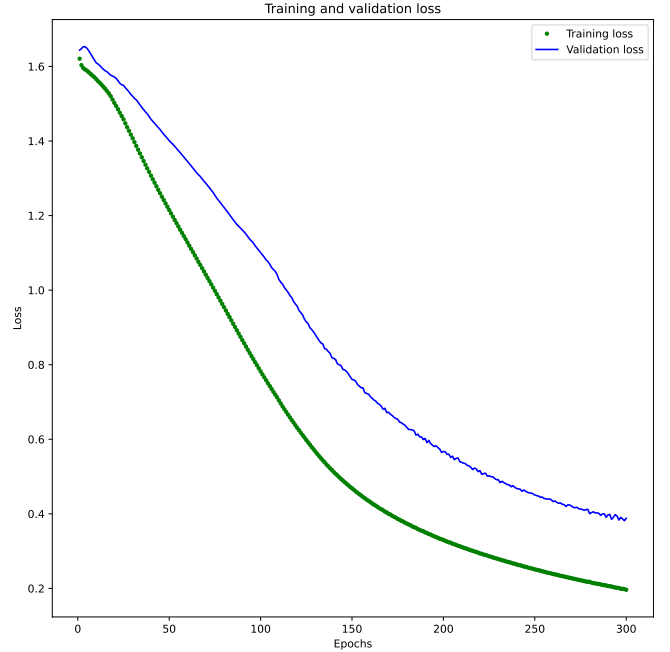


Figure 1: Training and validation loss over the 300 epochs.

6 Deployment on the Board

After the training, the Keras model is converted to the TensorFlow Lite format without quantisation, which gives 6900 bytes. A small function in the notebook writes this binary as a C array inside `scratch/model.h`, together with the class names and the number of classes, so the firmware always knows what each output of the network means. The firmware includes that file through `include/model.h`, which also produces a clear compilation error if the training has never been run.

On the board the model is executed by TensorFlow Lite Micro [2], with a static tensor arena of 8 kB. No dynamic memory is used at run time.

The output of the network is a vector of five probabilities. The firmware takes the highest one and applies two conditions before doing anything: the probability has to be at least 0.6, and the class has to have a key assigned to it. The four punches are mapped to the keys w (uppercut), d (jab), s (overhand) and a (hook), while the unknown class has no key at all. When both conditions hold, the board taps the key through its USB HID keyboard interface. Otherwise it stays silent, which is the behaviour we want for a movement that is not a punch or for a prediction the model is not sure about.

The repository also contains a test firmware. The notebook exports the test set as a second header file, and that firmware replays every test window through the model running on the device and compares the answer with the expected label. This is what confirms that the model behaves on the board in the same way as on the computer.

7 Results

At the last epoch the model reaches an accuracy of 0.9153 on the training set and of 0.8571 on the validation set. On the test set, which the model never saw during training, the accuracy is 0.8421, that is 16 correct answers out of 19.

Figure 2 shows the confusion matrix on the test set. The jab and the hook are always recognised correctly. Only three windows are wrong: one uppercut and one overhand are read as a hook, which are the punches that involve a rotation of the arm and therefore produce similar frequency content, and one window of the *unknown* class is read as an uppercut. This last mistake is the one that is felt by the user, because it is the only one that produces a key press that was not wanted.

Table 1 collects the cost of the system on the board.

Table 1: Cost of the system on the board

Quantity	Value
Model size (TensorFlow Lite)	6900 bytes
Tensor arena	8192 bytes
Window length	128 samples
Features per window	42
Flash used by the firmware	36.1% (355104)
RAM used by the firmware	24.3% (63760)

8 Limitations and Future Work

The main limitation is the size of the dataset. With 19 windows per class the test set is made of 19 windows only, so a single wrong answer moves the accuracy by more than five points, and the numbers above should be read as an indication and not as a precise measurement. All the data also comes from one person, so nothing can be said yet about a different user.

The second limitation is the set of punches. The cross had to be left out because it is too similar to the jab with the features used

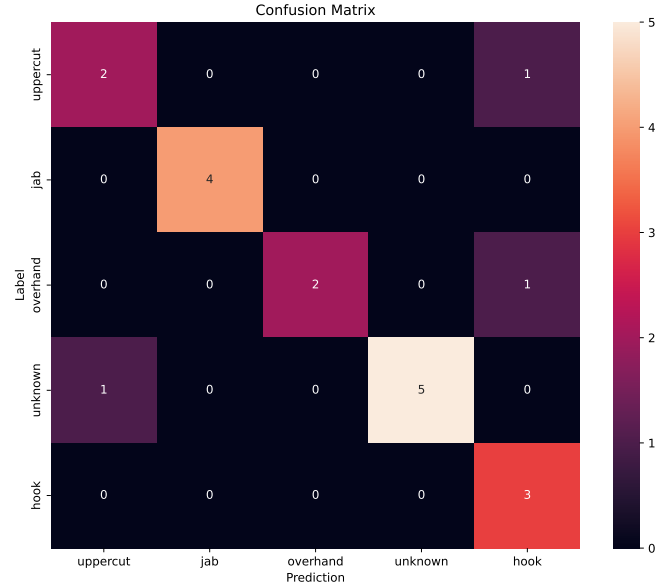


Figure 2: Confusion matrix on the 19 test windows. Rows are the true class, columns are the predicted class.

here. This could be solved by inserting another IMU on the other hand and sample both concurrently in order to differentiate also the hand that has issued the

The third limitation is practical. The board is worn on the hand and has to stay connected to the computer with a USB cable, which is exactly the wrong thing to have on a hand that is throwing punches. The cable limits the movement, and every fast punch pulls on it and on the connector of the board.

Future work follows from these points. The first step is to move the output from the wired USB HID interface to a BLE HID one, since the board already has Bluetooth Low Energy. The computer would still see a keyboard, but the hand would be free, the cable would not be pulled at every punch, and the board could run on a small battery. The second step is to record more data and from more people, and to add features that keep the direction of the movement, which is what the cross needs to be separated from the jab. The third step is to shorten the window to the real duration of a punch, which would cut the delay of the answer and would make a series of punches possible. This may also involve a change in the sensor choosing a faster one. As last and as previous mentioned, adding another IMU on the other hand could broaden the recognition capabilities of the system.

9 Conclusion

The project shows that a complete gesture recognition system fits on a single microcontroller: the sensor, the feature extraction, the neural network and the output interface all live on the same board, and the computer is only needed for the training. Computing the features on the device, and using the very same code when the dataset is recorded and when the model runs, is what finally made the recognition work on the board.

Acknowledgments

Titien Jacques Marie contributed to the first version of the data collection and of the training notebook.

References

- [1] Arduino. 2024. Arduino Nano 33 BLE Sense Product Reference Manual. <https://docs.arduino.cc/hardware/nano-33-ble-sense>.
- [2] Robert David, Jared Duke, Advait Jain, Vijay Janapa Reddi, Nat Jeffries, Jian Li, Nick Kreeger, Ian Nappier, Meghna Natraj, Tiezhen Wang, Pete Warden, and Rocky Rhodes. 2021. TensorFlow Lite Micro: Embedded Machine Learning for TinyML Systems. In *Proceedings of Machine Learning and Systems*, Vol. 3. 800:820.
- [3] Kosme. 2023. arduinoFFT: Fast Fourier Transform library for Arduino. <https://github.com/kosme/arduinoFFT>. Version 2.0.4.
- [4] Sandeep Mistry and Don Coleman. 2019. TinyML on Arduino: Gesture Recognition Tutorial. <https://github.com/arduino/ArduinoTensorFlowLiteTutorials>.